CHAPTER 15

# Designing & Implementing a “Minute/Hour Counter”

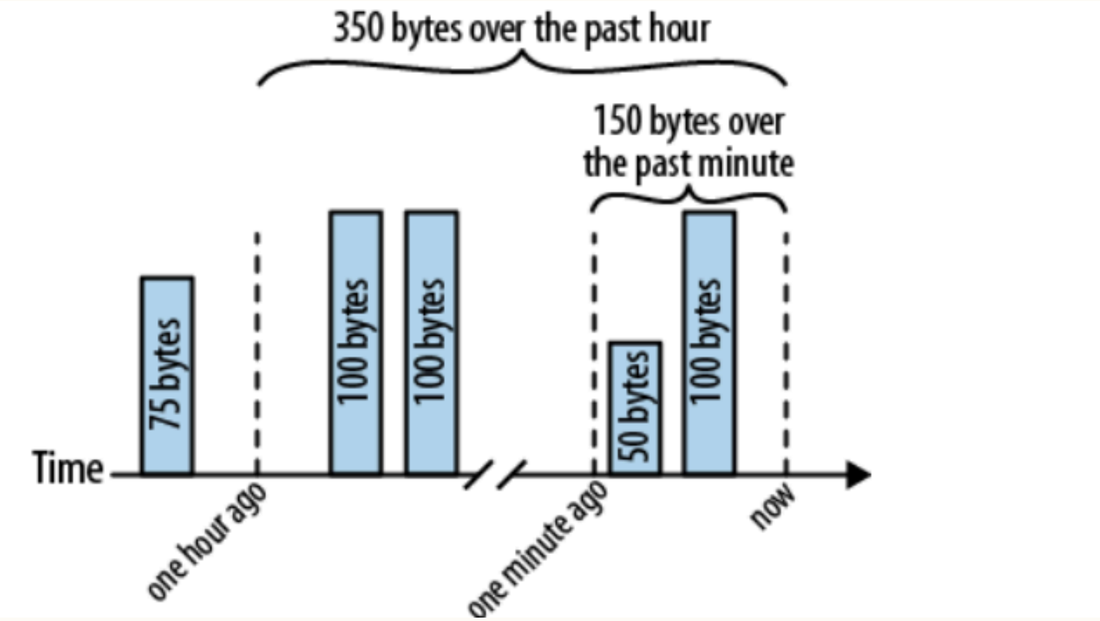

1. NY Chantharith e20220472
2. SOEUK Bondol e20221592
3. HOK Lyhour e20220759
4. CHOUB Botumraksa e20221709

THE PROBLEM

# Track the last minute and the last hour

01

**Bytes transferred by a web server —** past 60 seconds and past 3600 seconds.
**A sliding window, not clock time.**
**Simple to state.** Hard to make fast.




*150 bytes over the past minute · 350 over the past hour*

# Pick names before writing code

02

```
// --- final interface ---
void Add(int count);
int MinuteCount();
int HourCount();
```

**Count() → Add()** — Count is noun and verb; coworkers misread it.

**num_bytes → count** — keep it generic, someone may count queries.

**No “Get” prefix** — Get implies lightweight; this won’t be.

# Comments that kill ambiguity

03

**“over this minute” = two readings:** clock-minute vs. past 60 seconds.
**Rewrite it precisely:** “Return the count over the past 60 seconds.”
**State observable behavior:** “MinuteCount() will be larger by +count.”
**Test names on coworkers —** their first read is every reader’s first read.

ATTEMPT 1

# The naive solution

04

```
// --- state: every event ever seen ---
list<Event> events;   // (count, time)

// --- record one event ---
void Add(int count) {
    events.push_back(Event(count, time()));
}

// --- MinuteCount(): scan backward,
//     sum events from the last 60 seconds ---
```

**Correct.** But the loops are a mouthful.

**Duplicated complex code** — fix a bug twice, or miss one.

ATTEMPT 1, REVISED

# Refactor for the reader

05

```
// --- one helper for both queries ---
int CountSince(time_t cutoff) { ... }

int MinuteCount() { return CountSince(time() - 60);
}
int HourCount()   { return CountSince(time() -
3600); }
```

**One helper** replaces two near-identical loops.

**i → rit —** it's a reverse iterator; that fact is load-bearing.

**Keep for-loops standard-shaped** so readers skim them.

REALITY CHECK

# Readable — but broken on performance

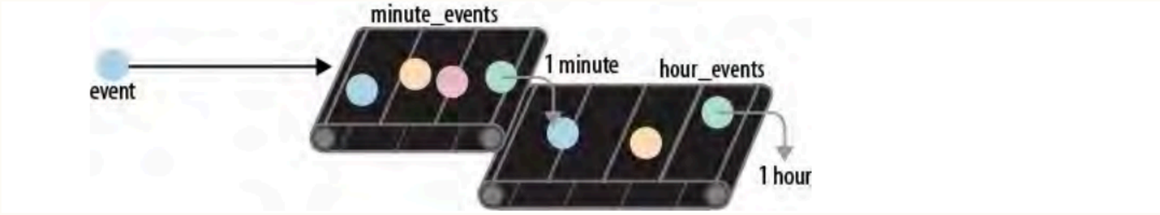
06

**Unbounded memory.** It keeps every event it has ever seen.
**O(n) queries.** 100 events/sec → one HourCount() scans ~3.6M events.
**Fix direction:** precomputed totals + auto-delete old data.

ATTEMPT 2

# The conveyor belt

07




```cpp
// Shift events from minute_events → hour_events and drop events > 1hr old
void ShiftOldEvents(time_t now_secs) {
  const int minute_ago = now_secs - 60;
  const int hour_ago = now_secs - 3600;

  // Move events > 1 minute old to hour_events
  while (!minute_events.empty() && minute_events.front().time ⩽ minute_ago) {
    hour_events.push_back(minute_events.front());
    minute_count -= minute_events.front().count;
    minute_events.pop_front();
  }

  // Remove events > 1 hour old
  while (!hour_events.empty() && hour_events.front().time ⩽ hour_ago) {
    hour_count -= hour_events.front().count;
    hour_events.pop_front();
  }
}
```

**New data arrives, old data "falls off".**

`ShiftOldEvents()` moves minute → hour lists.

**O(1) queries via precomputed `minute_count` / `hour_count`.**

**But: inflexible, and ~5 MB of unpredictable memory.**

ATTEMPT 3

# Time buckets: trade precision for performance

08

**60 one-second buckets** for the minute · 60 one-minute buckets for the hour.
**Accurate to 1 part in 60.** Fixed ~500 bytes of memory, forever.
**One task per class:** ConveyorQueue ▸ TrailingBucketCounter ▸ MinuteHourCounter.

THE VERDICT

# Longer code, easier reading

10

| Solution | Lines | Cost / HourCount( | Memory | Error |
|---|---|---|---|---|
| **Naive** | 33 | O(n) · ~3.6M | unbounded | 1 : 3600 |
| **Conveyor** | 55 | O(1) | ~5 MB | 1 : 3600 |
| **Bucketed** | 98 | O(1) | ~500 B | 1 : 60 |

*“100 lines that are all easy to read are better than 50 lines that aren’t.”*

— Chantharith

**Write naive first —** it exposes the real constraints.
**Polish names & comments —** test them on coworkers.
**One task per class.**